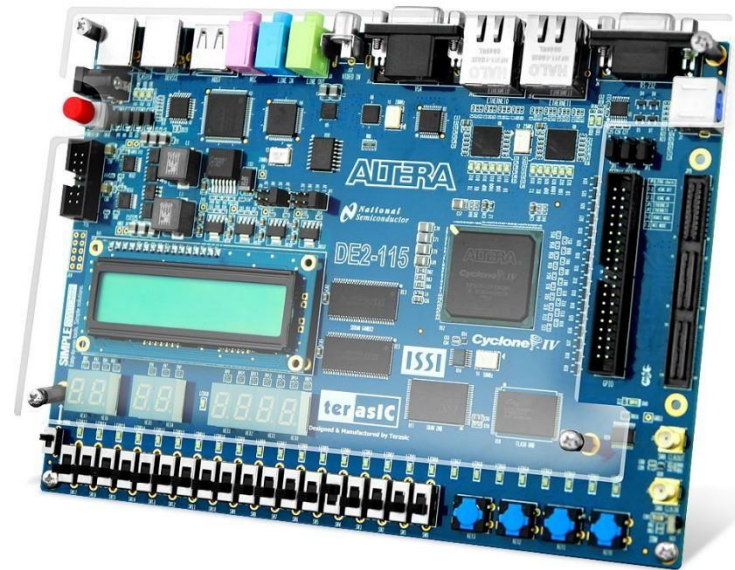


COEN-230

Digital System Design I

Dr. Fadi Alzhouri

Department of Electrical and Computer Engineering
Gust University





Textbooks (Copyrights):

- Digital Design: With an Introduction to the Verilog HDL, by M. Morris R. Mano and Michael D. Ciletti, 6 th Edition, Pearson, 2017.
- Fundamentals of Digital Logic with Verilog Design, by Stephen Brown and Zvonko Vranesic.



Outline

- Combinational circuits
- Circuits used to perform arithmetic operations
 - Half-adder circuit
 - Full-adder circuit
 - Ripple Carry Adder (RCA)
 - Carry Look-ahead Adder (CLA)
 - BCD numbers and adder



Combinational Circuits

- Combinational circuits: logic circuits in which the values of the outputs depend only on the **present values** of the inputs.
 - No feedback
 - No memory elements

Example:

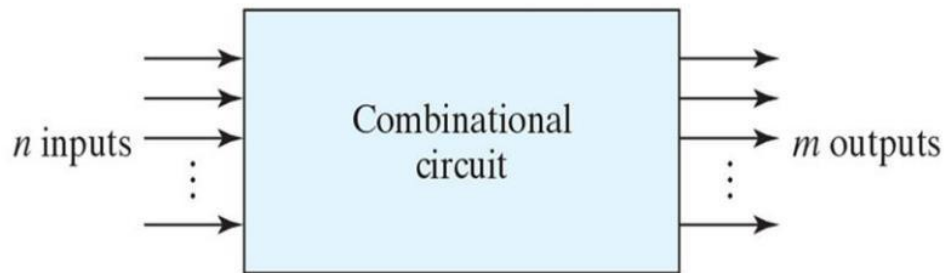
Circuits used to perform arithmetic operations

- Half-adder circuit
- Full-adder circuit
- Ripple Carry Adder (RCA)
- Carry Look-ahead Adder (CLA)
- BCD numbers and adder



Combinational Circuits (cont.)

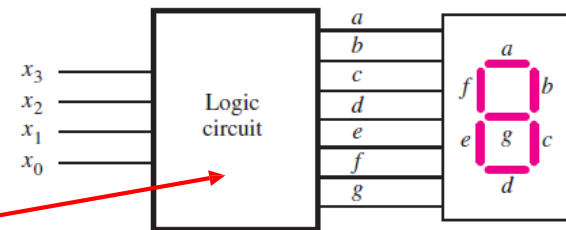
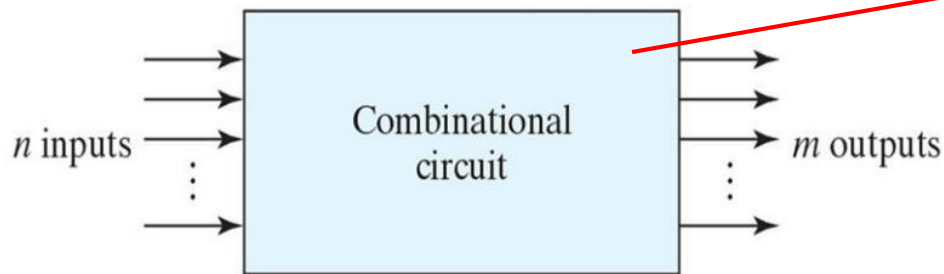
- A combinational circuit can be described using a truth table with 2^n rows.
- Each output will generate a column having a 1 or a 0 for each of the 2^n input choices.





Combinational Circuits (cont.)

- Example: Displaying BCD Numbers
 - $n = 4$
 - $m = 7$



(a) Logic circuit and 7-segment display

x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

(b) Truth table

What about the rest of the truth table?



Design steps.

- The design of a combinational circuit starts from the specifications of the design objectives.
- Determine the number of inputs and outputs.
- Derive the truth table showing the relation between inputs and each output.
- Derive the logical expression for each output.
- Simplify the Boolean expression of each output (using Boolean algebra or K-Map).
- Draw the circuit using a network of gates.
- Verify the correctness of the design (manually or by simulation).



Design Arithmetic circuits.

- Which Signed Representation?
- Arithmetic in sign & magnitude and 1's complement is not simple.
- Example: In 1's complement arithmetic, an extra addition may be needed to correct the result when there is a carry.

$$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array} \quad \begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array} \quad \begin{array}{r} 1010 \\ + 0010 \\ \hline 1100 \end{array}$$

$$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array} \quad \begin{array}{r} 0101 \\ + 1101 \\ \hline 10010 \\ \text{Carry } 1 \rightarrow \\ \hline 0011 \end{array}$$

$$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array} \quad \begin{array}{r} 1010 \\ + 1101 \\ \hline 10111 \\ \text{Carry } 1 \rightarrow \\ \hline 1000 \end{array}$$



2's Complement Addition

Regardless of operands, result is always correct

- Carry-out is ignored!
- Addition can be performed by an adder circuit.

$$\begin{array}{r} (+5) \\ + (+2) \\ \hline (+7) \end{array} \quad \begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ + (+2) \\ \hline (-3) \end{array} \quad \begin{array}{r} 1011 \\ + 0010 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} (+5) \\ + (-2) \\ \hline (+3) \end{array} \quad \begin{array}{r} 0101 \\ + 1110 \\ \hline 10011 \end{array}$$

↑
ignore

$$\begin{array}{r} (-5) \\ + (-2) \\ \hline (-7) \end{array} \quad \begin{array}{r} 1011 \\ + 1110 \\ \hline 11001 \end{array}$$

↑
ignore



2's Complement Subtraction

Easiest way is to negate the second term and then add it to the first.

Need to find the 2's complement of the 2nd term.

- Conclusion: Subtraction can be performed using an **adder** unit

$$\begin{array}{r} (+5) \\ - (+2) \\ \hline (+3) \end{array} \quad \begin{array}{r} 0101 \\ - 0010 \\ \hline \end{array} \quad \Rightarrow \quad \begin{array}{r} 0101 \\ + 1110 \\ \hline \cancel{1}0011 \end{array}$$

$$\begin{array}{r} (-5) \\ - (+2) \\ \hline (-7) \end{array} \quad \begin{array}{r} 1011 \\ - 0010 \\ \hline \end{array} \quad \Rightarrow \quad \begin{array}{r} 1011 \\ + 1110 \\ \hline \cancel{1}1001 \end{array}$$

$$\begin{array}{r} (+5) \\ - (-2) \\ \hline (+7) \end{array} \quad \begin{array}{r} 0101 \\ - 1110 \\ \hline \end{array} \quad \Rightarrow \quad \begin{array}{r} 0101 \\ + 0010 \\ \hline 0111 \end{array}$$

$$\begin{array}{r} (-5) \\ - (-2) \\ \hline (-3) \end{array} \quad \begin{array}{r} 1011 \\ - 1110 \\ \hline \end{array} \quad \Rightarrow \quad \begin{array}{r} 1011 \\ + 0010 \\ \hline 1101 \end{array}$$



Addition of Binary Numbers

- How do we perform addition of two, 1-bit numbers?

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \ 0 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 0 \ 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 0 \ 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 1 \ 0 \end{array}$$

$$\begin{array}{r} x \\ + y \\ \hline c \ s \end{array}$$

Carry $\xrightarrow{\quad}$ $\uparrow$ $\uparrow$ Sum

- Truth table?
- Logic circuit?

$x \quad y$		Carry c	Sum s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

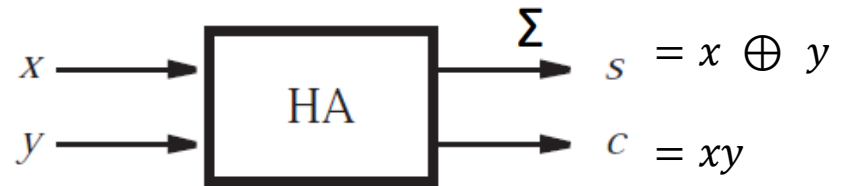
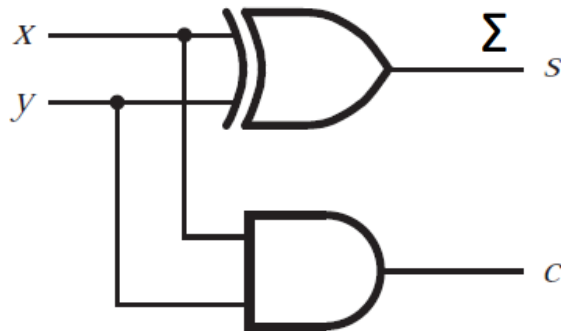
$$s = x'y + xy' \quad \text{or} \quad s = x \oplus y$$

$$c = xy$$



Half Adder Implementation

- Half-Adder (HA): a combinational logic circuit with two inputs and two outputs (the sum bit and the carry bit).
- HA assumes we are adding a **single** bit



What if we have **multiple** bits? Can we use HAs to do addition?



Add n-bit Numbers

For larger numbers it is still necessary to add each pair of bits, but for each bit position i , the addition operation may include a *carry-in* from bit position $i - 1$.

$$\begin{array}{rcl} X = x_4x_3x_2x_1x_0 & 0\ 1\ 1\ 1\ 1 & (15)_{10} \\ + Y = y_4y_3y_2y_1y_0 & 0\ 1\ 0\ 1\ 0 & (10)_{10} \\ \hline & 1\ 1\ 1\ 0 & \leftarrow \text{Generated carries} \\ \hline S = s_4s_3s_2s_1s_0 & 1\ 1\ 0\ 0\ 1 & (25)_{10} \end{array}$$

- Essentially we need to add the bits x_i and y_i along with the previous carry (c_i) to get new carry c_{i+1} and sum bit s_i



Full-Adder Implementation

- Full-Adder (FA): a combinational circuit with three inputs (one bit from a number, one bit from another number, and a carry-in bit) and two outputs (the sum bit and the carry-out bit).
- Truth table and Karnaugh maps:

c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Note that $s_i = (x_i' y_i + x_i y_i') c_i' + (x_i' y_i' + x_i y_i) c_i$
 $= (x_i \oplus y_i) c_i' + (x_i \oplus y_i)' c_i$
 $= (x_i \oplus y_i) \oplus c_i$

$c_i \backslash x_i y_i$	00	01	11	10
0		1		1
1	1		1	

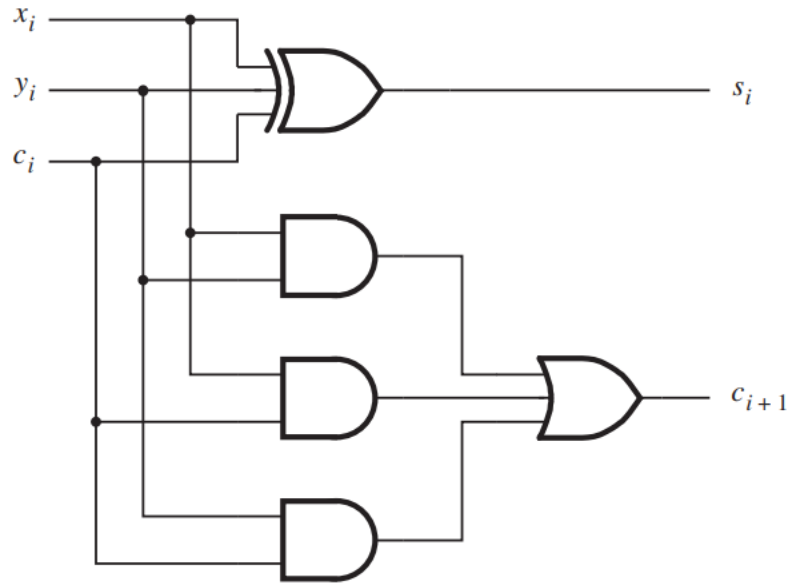
$$s_i = x_i \oplus y_i \oplus c_i$$

$c_i \backslash x_i y_i$	00	01	11	10
0			1	
1		1	1	1

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$



Full-Adder Circuit

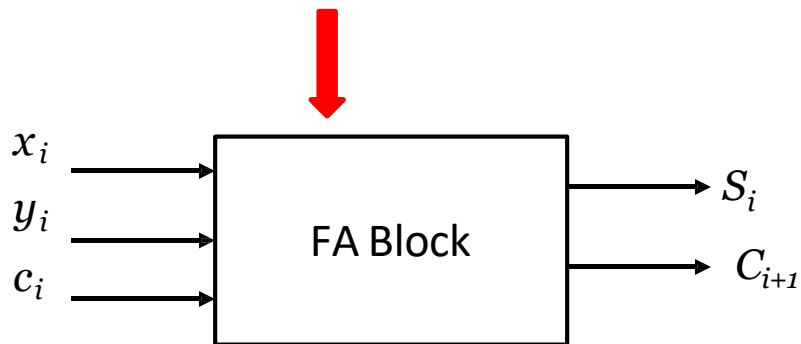


$c_i \backslash x_i y_i$				
	00	01	11	10
0		1		1
1	1		1	

$$s_i = x_i \oplus y_i \oplus c_i$$

$c_i \backslash x_i y_i$				
	00	01	11	10
0			1	
1		1	1	1

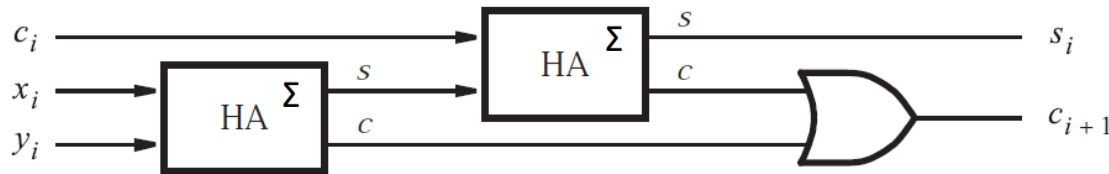
$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$



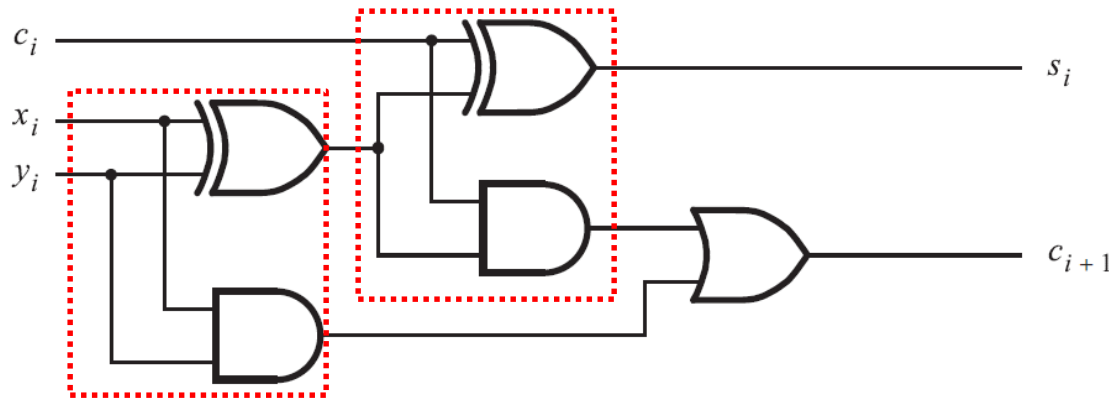


Decomposed full adder

Design a Full adder (FA) using Half adders (HAs).



(a) Block diagram



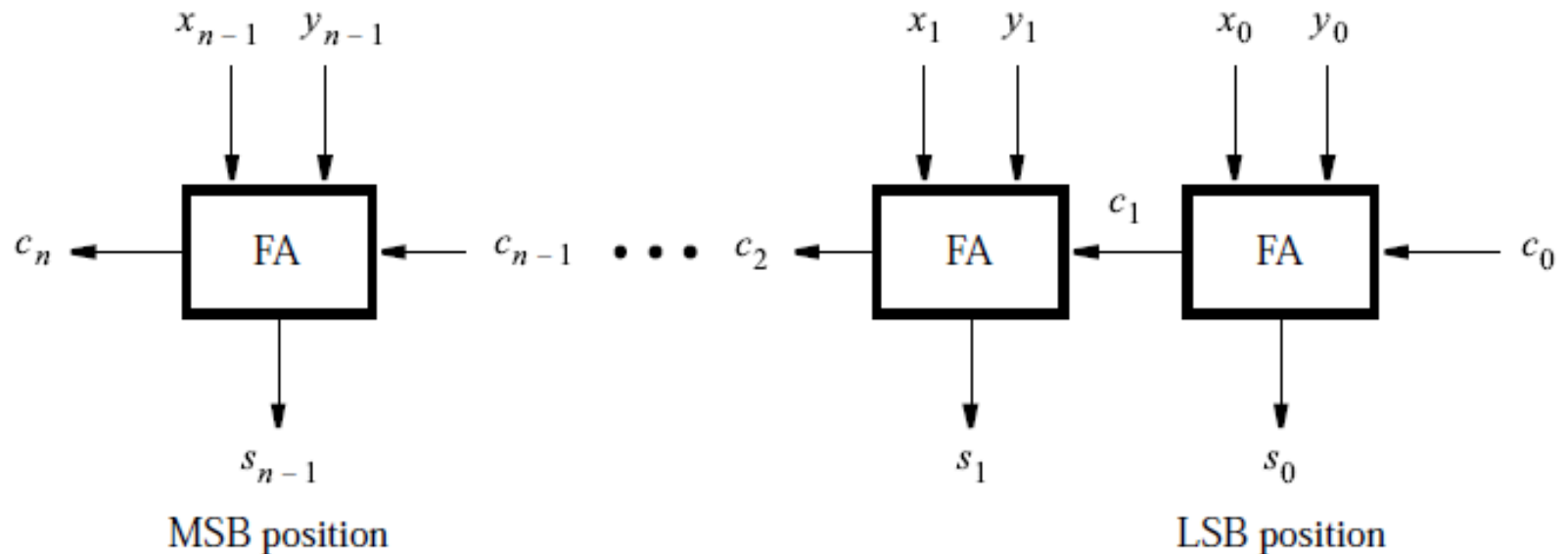
(b) Detailed diagram

● Why does it work?



N-bit Ripple-Carry Adder

- An n-bit adder can be constructed by connecting “n” FAs in series so that the carry “ripples” from one to the next.
- It is used to add two n-bit numbers.

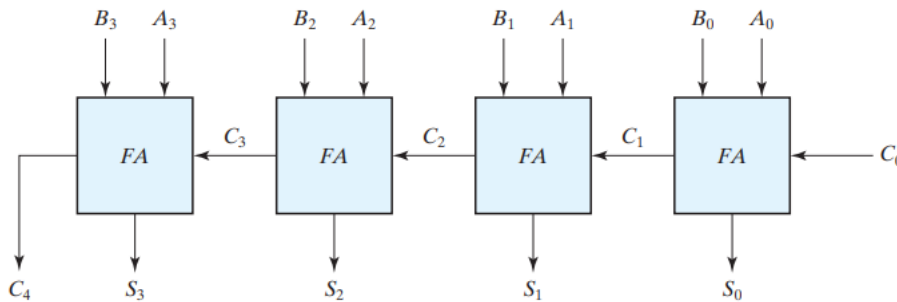


- What is the problem with this implementation?

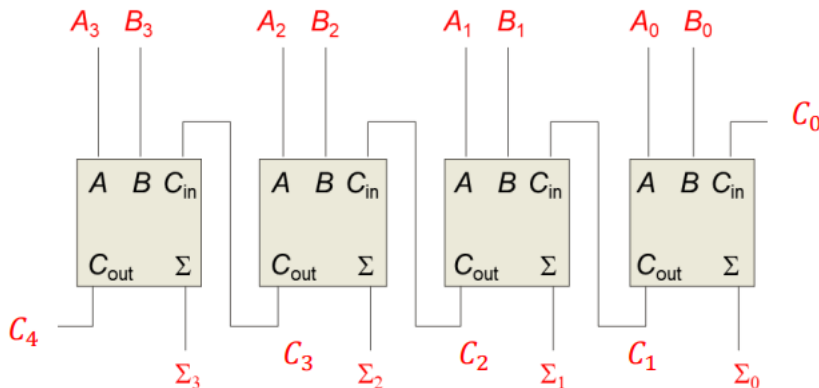


Example: 4-bit Ripple-Carry Adder

- Design a 4-bit ripple carry adder using FA blocks.



OR



By Dr. Soleimani



2's Complement Subtraction (circuit)

- For the subtraction ($X - Y$) it is necessary to use the 2's complement of Y .
- A 2's complement can be obtained by adding 1 to the 1's complement of Y .

So, how do we complement a number?

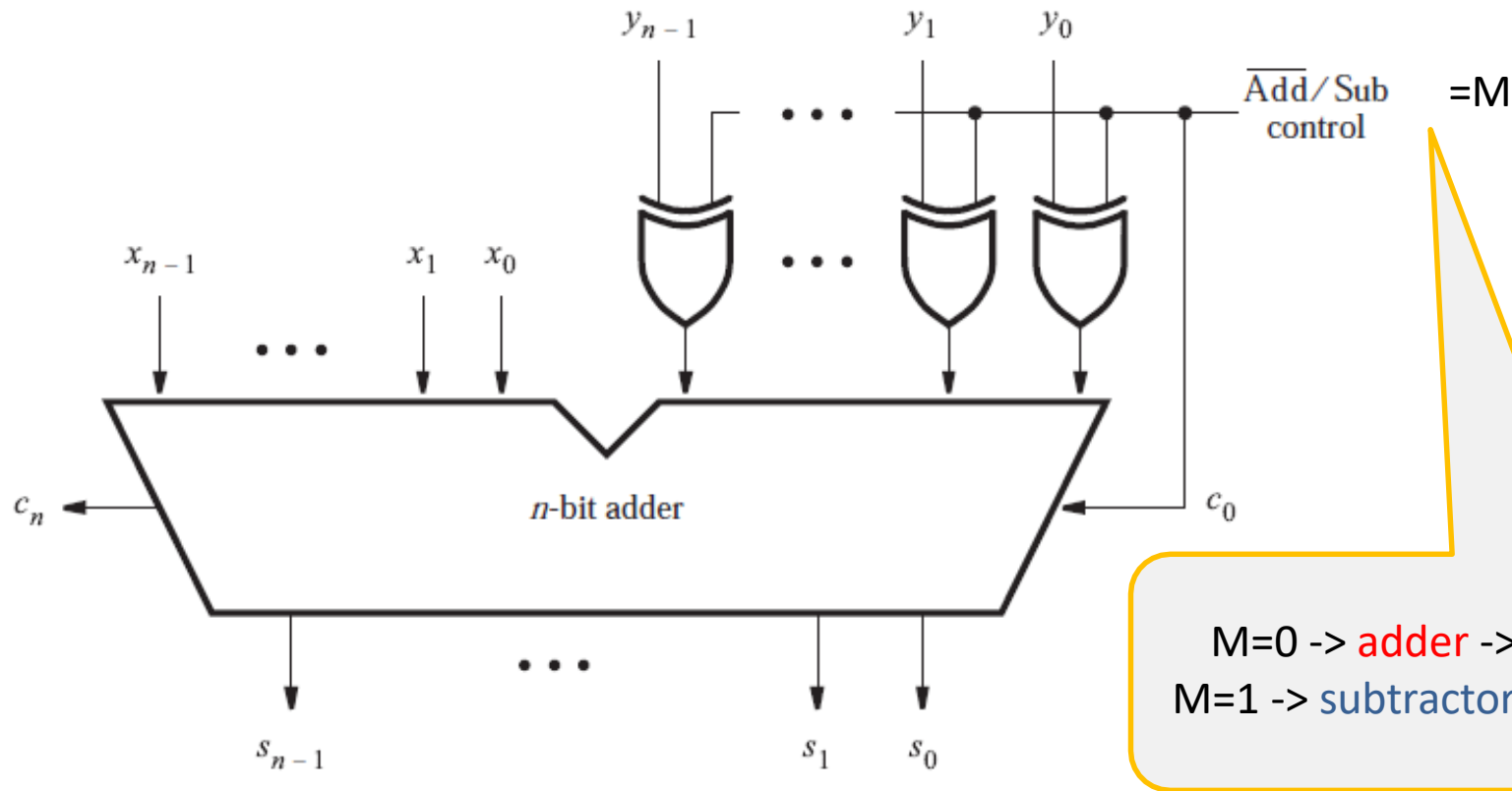
Adding 1 in the least-significant bit position can be accomplished simply by setting the carry-in bit c_0 to 1.

● We'll use an XOR or NOT

- A 2-input XOR gate can be thought of as using one input as a **control signal** that determines whether the true or complemented value of the other input will be as the output value.
- Look at $x \oplus y = x'y + xy'$.
 - if $x = 0$, then y is the output.
 - But if $x = 1$, the output will be y' .



Adder and Subtractor Unit



Design approach:

Design circuits to be as flexible as possible by exploiting common portions of circuits for as many tasks as possible. This approach minimizes the number of gates needed and reduces the wiring complexity substantially.



Arithmetic overflow

- The result of addition or subtraction must fit within the significant bits used to represent the numbers.

If the result does not fit in this range, then we say that *arithmetic overflow* has occurred.

Must be able to detect the occurrence of overflow (basically occurs when numbers have the same sign)

- Can be detected using the carry-out from the MSB position, and from the sign-bit position.
- XOR the last two Carry's will detect the overflow

$$\text{Overflow} = c_{n-1} \oplus c_n$$

If $OV = 0$, No overflow , the result is correct

If $OV = 1$, there is an overflow and the result is not correct.



Example: Arithmetic Overflow

$$\begin{array}{r} (+7) \quad 0111 \\ + (+2) \quad +0010 \\ \hline (+9) \quad 1001 \\ c_4 = 0 \\ c_3 = 1 \end{array}$$

$$\begin{array}{r} (-7) \quad 1001 \\ + (+2) \quad +0010 \\ \hline (-5) \quad 1011 \\ c_4 = 0 \\ c_3 = 0 \end{array}$$

$$\begin{array}{r} (+7) \quad 0111 \\ + (-2) \quad +1110 \\ \hline (+5) \quad 10101 \\ c_4 = 1 \\ c_3 = 1 \end{array}$$

$$\begin{array}{r} (-7) \quad 1001 \\ + (-2) \quad +1110 \\ \hline (-9) \quad 10111 \\ c_4 = 1 \\ c_3 = 0 \end{array}$$

An overflow may occur if the added two numbers are both positive or both negative, but cannot occur if a positive number is added to a negative number

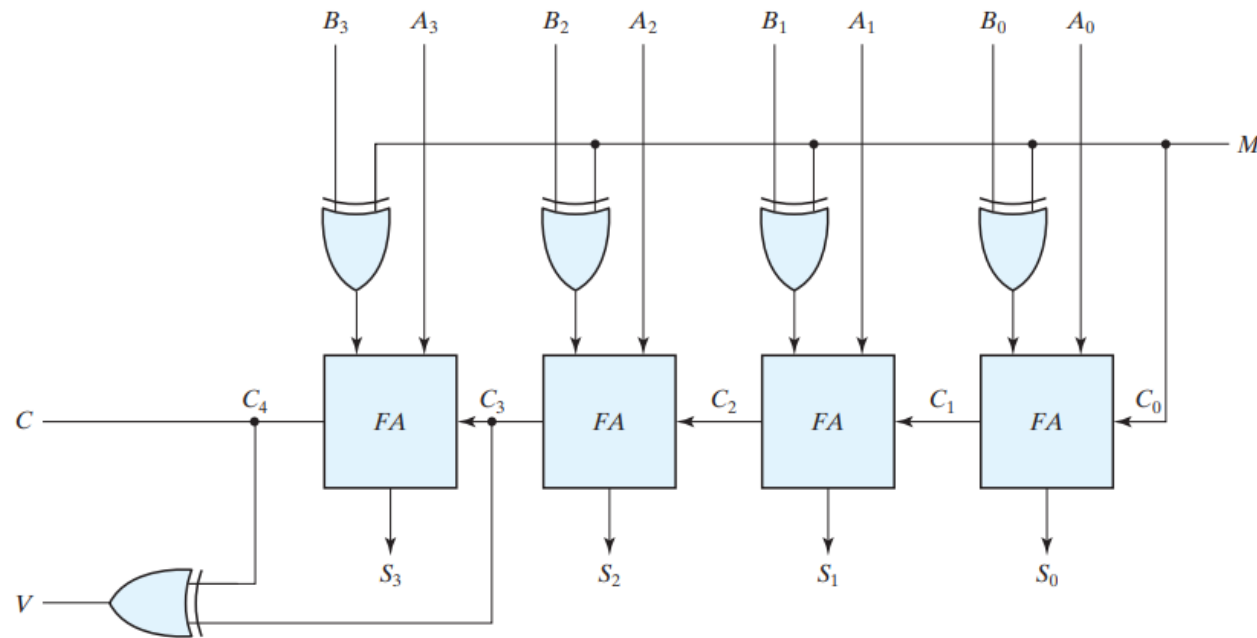
$$\begin{aligned} \text{Overflow} &= c_3 \bar{c}_4 + \bar{c}_3 c_4 \\ &= c_3 \oplus c_4 \end{aligned}$$

and for n-bit numbers

$$\text{Overflow} = c_{n-1} \oplus c_n$$



Adder and Subtractor with overflow



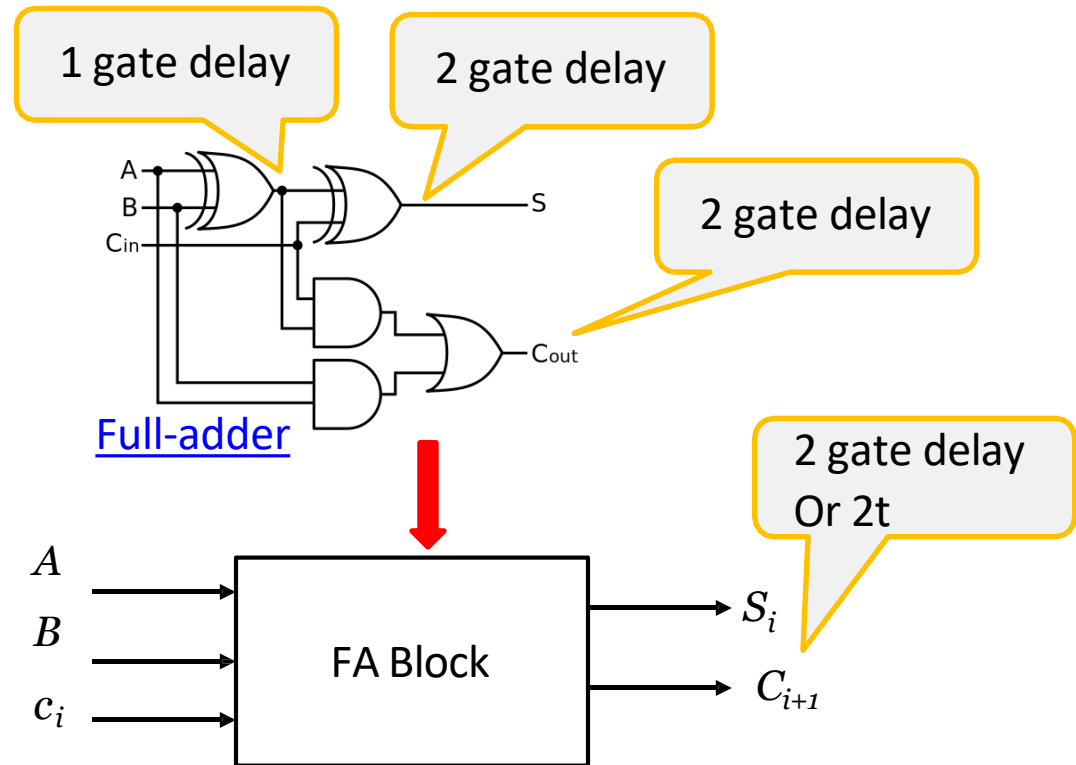
4-bit adder/subtractor with overflow detection

$$V \text{ (or OV)} = c_4 \oplus c_3$$



Performance issues

Addition and subtraction of numbers are fundamental operations. Speed of these operations has a strong impact on the overall performance of a computer. Speed is limited by the longest delay along the paths through the circuit (**Critical Path**)

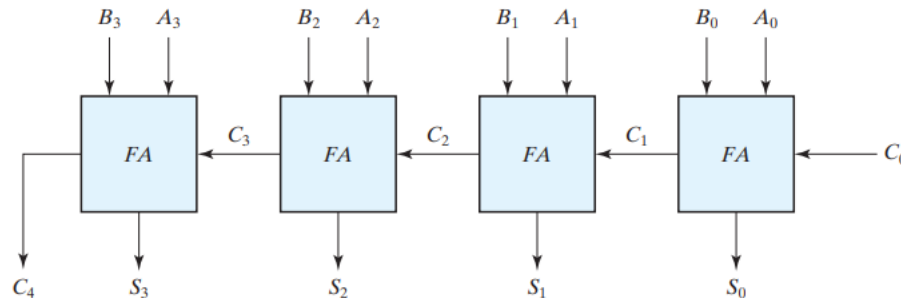




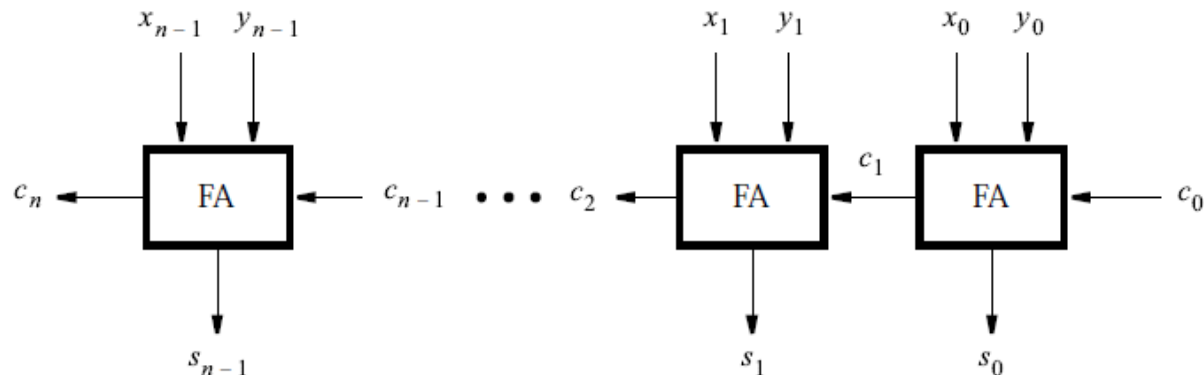
Performance issues

Given a 4-bit ripple carry adder, calculate the delay of the critical path.

C_4 is available after
4 x 2 gate delay



If carry-out of the FA takes 2 gate delays ($2t$), the **total delay is almost $nx2t$** ; unacceptably poor performance for large n .





n-bit ripple carry adder using gates

Design a 4-bit ripple carry adder using gates.

$$S_i = x_i \oplus y_i \oplus c_i$$

$$\begin{aligned} C_{i+1} &= x_i y_i + x_i c_i + y_i c_i \\ &= x_i y_i + (x_i + y_i) c_i \\ &= g_i + p_i c_i \end{aligned}$$

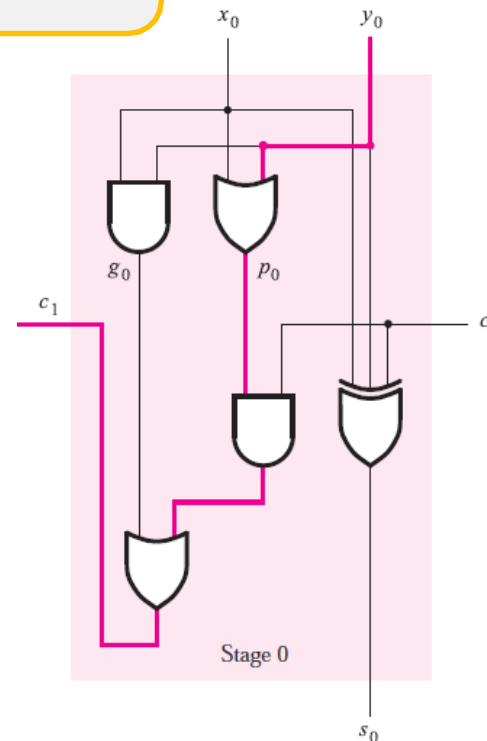
Assume $g_i = x_i y_i$,
 $p_i = (x_i + y_i)$

$x_i y_i$ c_i	00	01	11	10
0		1		1
1	1		1	

$$s_i = x_i \oplus y_i \oplus c_i$$

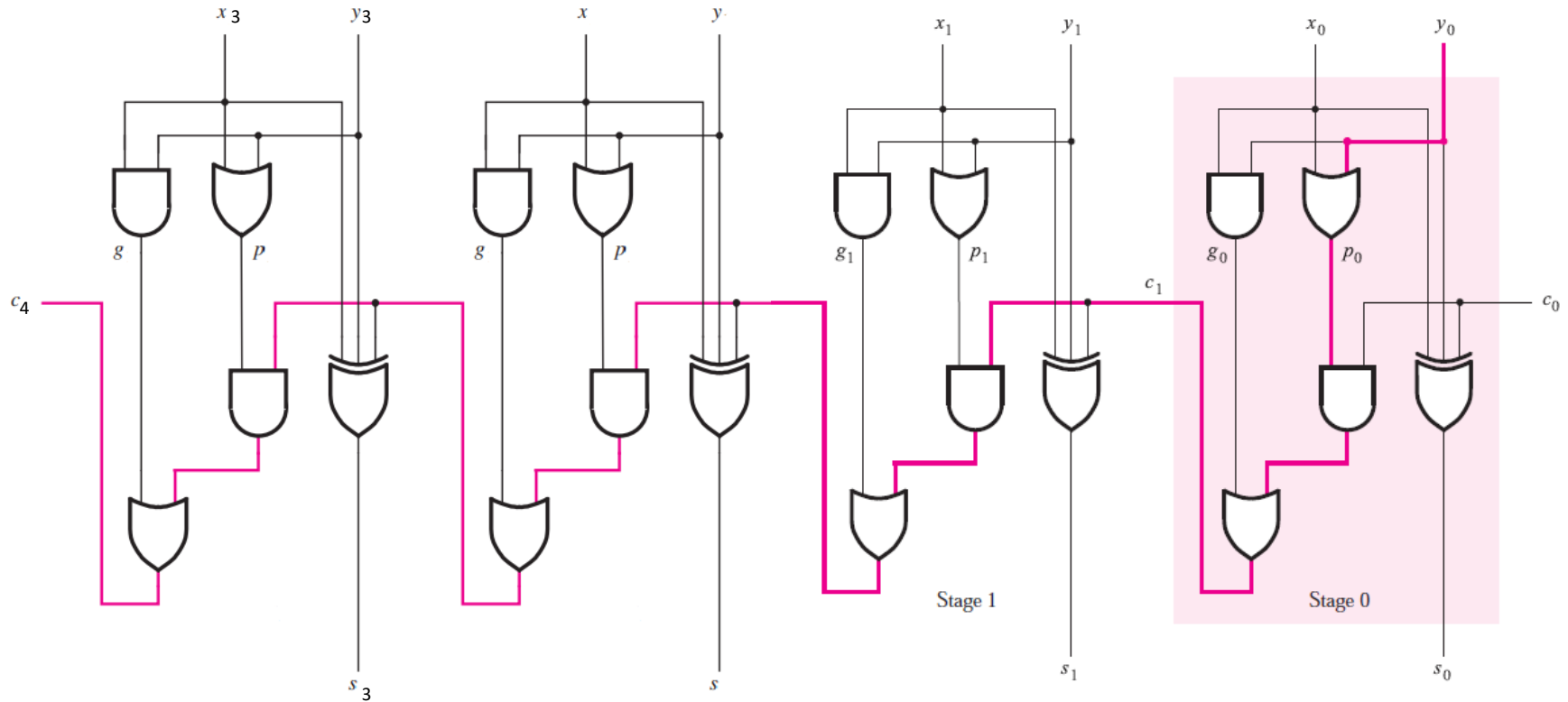
$x_i y_i$ c_i	00	01	11	10
0			1	
1		1	1	1

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$





4-bit ripple carry adder using gates



Analyze the performance of this adder by finding the delay in producing each signal in terms of gate delay.



Carry Look-Ahead Adder (CLA)

- Idea: consider how the carry bit is generated?

Need 2 1's to generate carry and at least one to propagate carry

Define **generate** a carry_i as $g_i = x_i y_i$

Define **propagate** a carry_i as $p_i = x_i + y_i$

- CLA is used to improve the performance by reducing the time delay
- Therefore, for a 4-bit adder, the carries are

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_1 \rightarrow c_2 = g_1 + p_1 g_0 + p_1 p_0 c_0$$

$$c_3 = g_2 + p_2 c_2 \quad c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

$$c_4 = g_3 + p_3 c_3 \quad c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

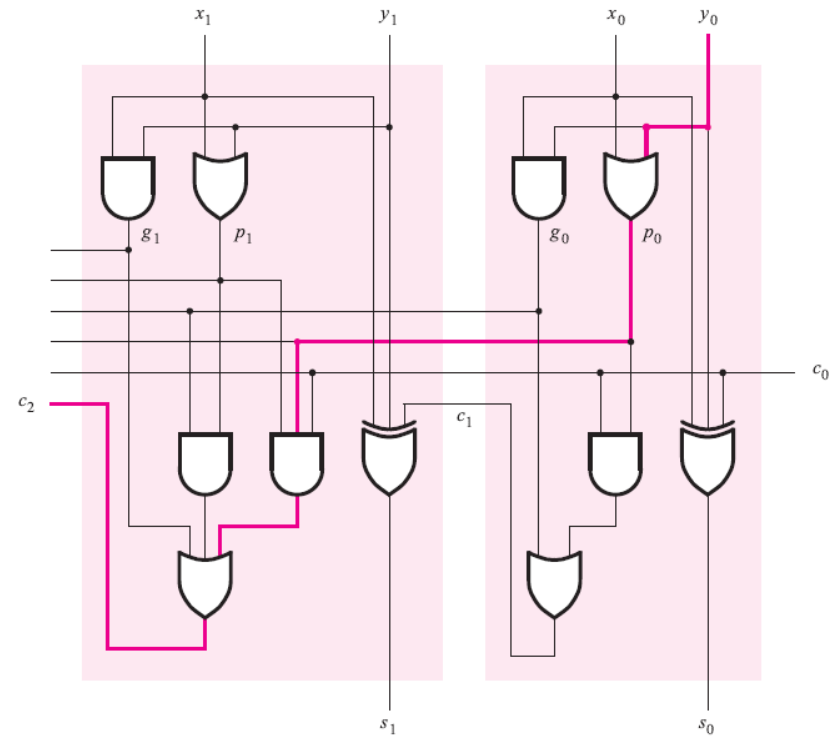
} 2 level
AND/OR

- Generalizing carry in the CLA:

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$



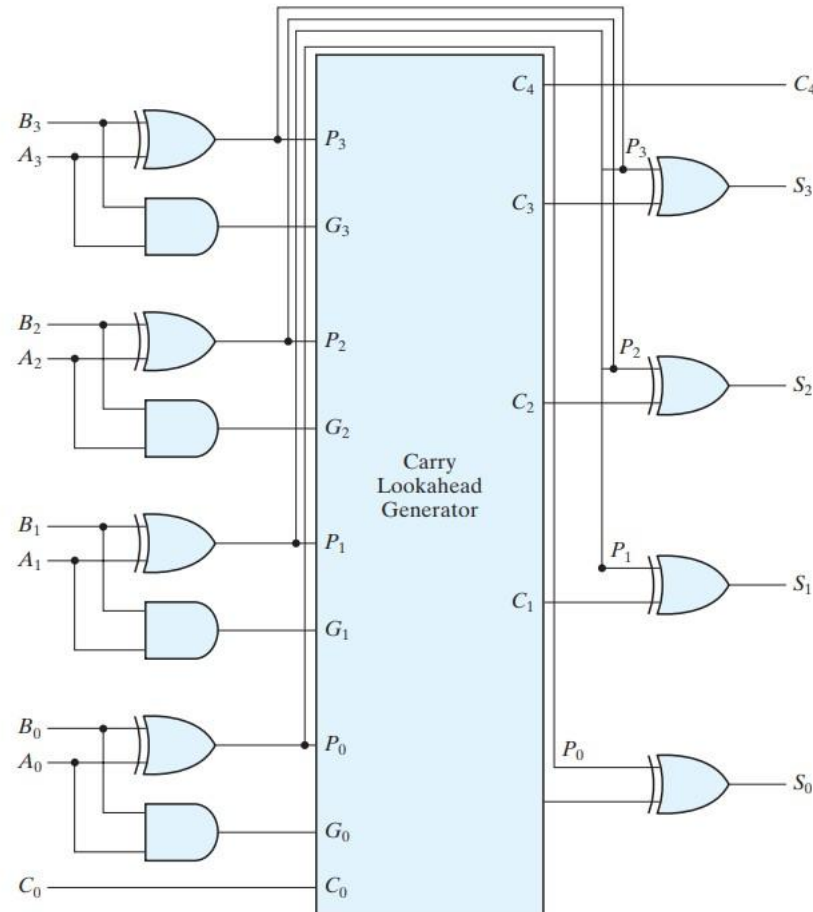
4-bit carry look-ahead adder using gates



Complete the design and analyze it for delays in producing all signals.



4-bit carry look-ahead adder using gates



Carry look-ahead block implements:

$$c_{i+1} = g_i + p_i g_{i-1} + p_i p_{i-1} g_{i-2} + \cdots + p_i p_{i-1} \cdots p_2 p_1 g_0 + p_i p_{i-1} \cdots p_1 p_0 c_0$$



Alternative approach (Full adder without carry out)

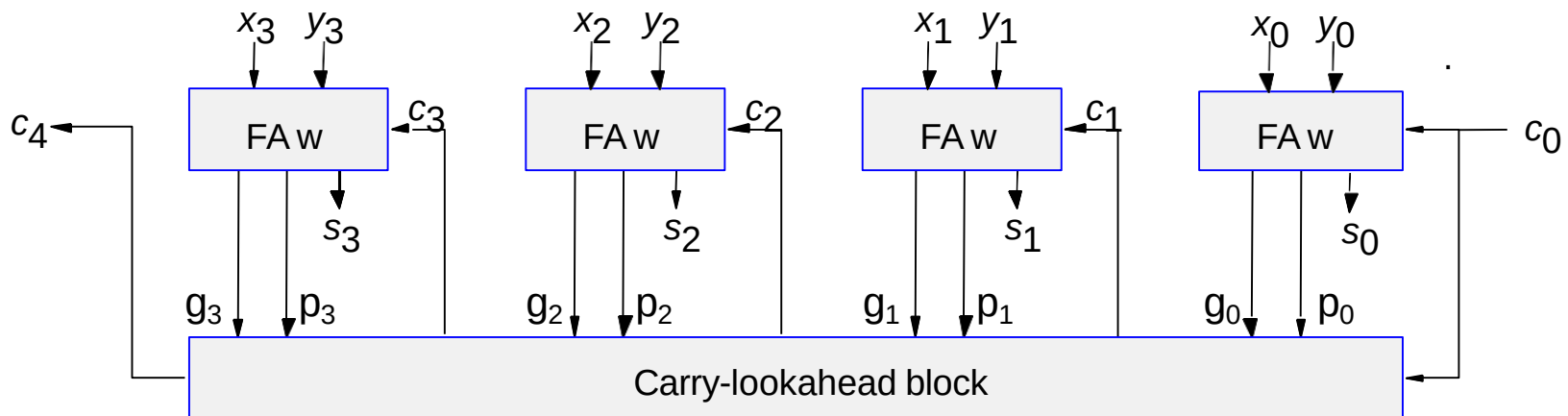
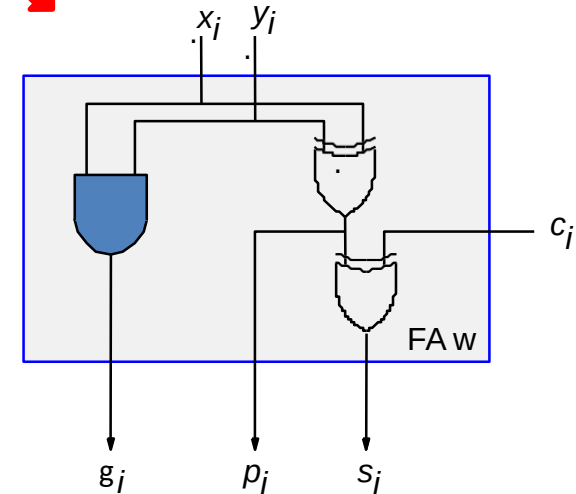
All p_i and g_i generated in parallel

After 1 gate delay of input application

Simpler circuit for 1bit adder

Use XOR to generate p_i

Don't care about case when both inputs are 1 $\rightarrow$
 g_i will be set !!





Binary Coded Decimals (BCD)

Binary representation is the most appropriate for digital systems

However people prefer to deal with decimal numbers
Especially when interfacing with outside world

To ease handling of decimal numbers, they are represented as a string of bits

In particular a string of 4 bits

To allow representation of digits 0...9

Most appropriate representation is using binary value to represent decimal number

Hence name Binary Coded Decimal (BCD)



BCD Representation

Table shows encoding

Note that not all combinations are used

1010 and above are not used >9

These combinations represent an invalid BCD number or don't care in the truth table

Maximum BCD number

Decimal #	BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001



BCD Digits Addition

Using a binary adder the result may not be a BCD number and therefore the result must be adjusted.

Add 6=0110

- Case 1: the result > 9

$$\begin{array}{r} X \\ + Y \\ \hline Z \end{array} \quad \begin{array}{r} 0111 \\ + 0101 \\ \hline 1100 \\ + 0110 \\ \hline \text{carry} \rightarrow 10010 \\ \underbrace{\hspace{1.5cm}} \\ S = 2 \end{array} \quad \begin{array}{r} 7 \\ + 5 \\ \hline 12 \end{array}$$

If $Z > 9$, then $\text{Sum} = Z + 6$
Else $\text{Sum} = Z$

- Case 2: the carry = 1

$$\begin{array}{r} X \\ + Y \\ \hline Z \end{array} \quad \begin{array}{r} 1000 \\ + 1001 \\ \hline 10001 \\ + 0110 \\ \hline \text{carry} \rightarrow 10111 \\ \underbrace{\hspace{1.5cm}} \\ S = 7 \end{array} \quad \begin{array}{r} 8 \\ + 9 \\ \hline 17 \end{array}$$



- One is used to add the two BCD numbers
- The second is used to adjust the result when needed

